

□ ANDRYUS LAB

Documento Maestro de Arquitectura y Ejecución — v1.0

□ **Una plataforma modular de microherramientas digitales, diseñada para resolver problemas concretos con mínimo almacenamiento, bajo coste operativo y datos efímeros cuando sea posible.**

Propósito del documento

Servir como especificación conceptual y técnica para que un desarrollador, ingeniero o sistema de IA pueda comprender qué es Andryus Lab, cómo debe construirse, qué componentes utilizar, cómo administrar los datos y qué principios no deben romperse.

Idea central: “No almacenar lo que puede calcularse nuevamente; no conservar lo que puede expirar; no subir al servidor lo que puede procesarse en el navegador.”

□ Documento preparado: 21 de septiembre de 2026

1. Visión y objetivo

Andryus Lab será una plataforma web modular compuesta por pequeñas herramientas independientes. Cada herramienta resolverá una necesidad concreta: calcular, convertir, diagnosticar, generar, comparar, transformar o compartir información de manera temporal.

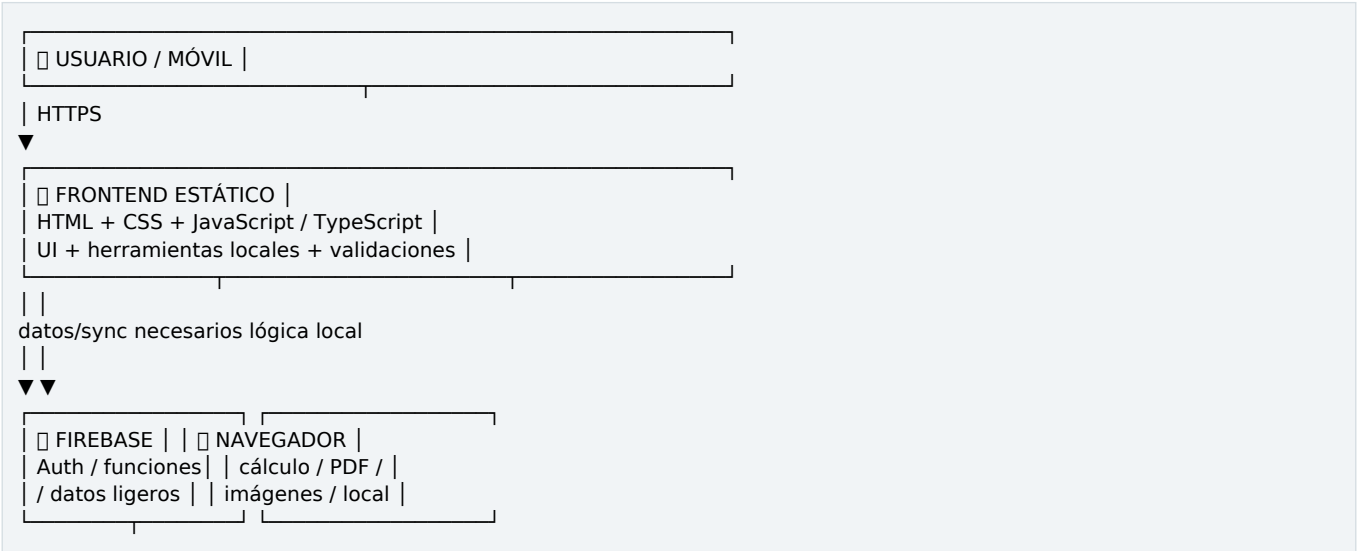
El objetivo técnico no es construir una aplicación monolítica que dependa de una gran base de datos. El objetivo es crear un ecosistema donde el navegador haga la mayor cantidad de trabajo posible y donde el backend solo intervenga cuando aporte valor real.

- ⚡ Respuesta inmediata y experiencia móvil-first.
- ♻ Almacenamiento mínimo y, cuando sea posible, temporal.
- ☑ Lógica de negocio ejecutada en cliente para herramientas puramente computacionales.
- ☑ Backend reservado para autenticación, sincronización, datos compartidos y funciones que realmente lo necesiten.
- ☑ Arquitectura preparada para monetización futura sin convertir la infraestructura gratuita en una dependencia crítica.
- ☑ Cada herramienta debe poder desarrollarse, probarse y desplegarse de forma modular.

2. Principios arquitectónicos

#	Principio	Regla
1	Local-first	Si una operación puede hacerse con JavaScript en el dispositivo, se hace localmente.
2	Data minimization	Solo se almacena el mínimo dato imprescindible.
3	Ephemeral by default	Los datos temporales tienen TTL y deben eliminarse automáticamente.
4	No heavy storage	Evitar usar la nube como depósito de PDFs, imágenes o vídeos cuando el archivo pueda generarse o procesarse localmente.
5	Provider independence	Firebase y Supabase son componentes intercambiables; la aplicación no debe quedar conceptualmente atada a uno solo.
6	Modularity	Cada herramienta es un módulo con entrada, procesamiento, salida y pruebas propias.

3. Arquitectura general



| datos estructurados / APIs

▼
□ SUPABASE |
PostgreSQL |
Edge Functions |
Storage puntual |

△ No es obligatorio utilizar Firebase y Supabase simultáneamente. La arquitectura debe permitir que una herramienta use solo frontend, frontend + Firebase, frontend + Supabase o una combinación controlada cuando exista una razón técnica.

4. □ Capas del sistema

□ Presentación — Componentes visuales, navegación, formularios, accesibilidad, responsive design y estados de carga/error.

□ Dominio — Funciones puras que reciben datos y devuelven resultados. Idealmente independientes del framework y del backend.

□ Persistencia — Adaptadores para LocalStorage/IndexedDB, Firebase o Supabase. La lógica de negocio no debe depender directamente de SDKs.

□ Seguridad — Autenticación, autorización, validación, reglas de acceso, rate limiting y sanitización.

□ Retención — TTL, expiración, limpieza, jobs periódicos y políticas de borrado.

□ Observabilidad — Errores, métricas y eventos mínimos, evitando recopilar datos innecesarios.

5. □ Estructura de código propuesta

```
andryus-lab/  
├── apps/  
│   ├── web/  
│   └── src/  
│       ├── app/  
│       ├── components/  
│       ├── tools/  
│       │   ├── price-calculator/  
│       │   ├── battery-calculator/  
│       │   ├── qr-generator/  
│       │   └── temporary-share/  
│       ├── services/  
│       │   ├── firebase/  
│       │   └── supabase/  
│       ├── storage/  
│       ├── security/  
│       ├── utils/  
│       └── tests/  
├── packages/  
│   ├── ui/  
│   ├── domain/  
│   ├── schemas/  
│   └── config/  
├── docs/  
└── README.md
```

La separación clave es tools → domain → services: una herramienta no debería contener consultas directas a Firebase o Supabase mezcladas con su cálculo.

6. ♻ Ciclo de vida de los datos temporales

CREAR
↓
VALIDAR

↓
GUARDAR SOLO LO NECESARIO
↓
ASIGNAR TTL / expiresAt
↓
UTILIZAR
↓
EXPIRAR O ELIMINAR MANUALMENTE
↓
LIMPIAR ÍNDICES / REFERENCIAS
↓
ESPACIO REUTILIZABLE

Ejemplo de registro temporal:

```
{  
  "id": "7F92KA",  
  "payload": "...",  
  "createdAt": "2026-09-21T22:00:00Z",  
  "expiresAt": "2026-09-21T22:10:00Z",  
  "maxViews": 1,  
  "views": 0  
}
```

La aplicación debe comprobar expiresAt antes de entregar el contenido. La limpieza física del registro debe realizarse mediante un proceso programado o mecanismo de expiración del proveedor cuando esté disponible. El TTL es una política de negocio; no debe asumirse que borrar un registro equivale a recuperar instantáneamente todas las cuotas del proveedor.

7. Contrato técnico de una herramienta

```
type ToolDefinition = {  
  id: string;  
  name: string;  
  category: string;  
  version: string;  
  
  inputSchema: Schema;  
  outputSchema: Schema;  
  
  execute(input: unknown): ToolResult;  
  
  persistence: "none" | "local" | "remote";  
  ttlSeconds?: number;  
  
  requiresAuth: boolean;  
  supportsOffline: boolean;  
};
```

Cada herramienta debería declarar explícitamente si necesita persistencia. Por defecto: persistence = none.

8. Ejemplo: calculadora de precio real

```
function calculateRealCost(input) {  
  const {  
    purchasePrice,  
    shipping,  
    commission,  
    taxes,  
    packaging,  
    advertising  
  } = input;  
  
  const total =  
    purchasePrice +  
    shipping +  
    commission +
```

```
taxes +
packaging +
advertising;

return {
totalCost: total,
currency: input.currency
};
}
```

Esta función es ideal para ejecutarse 100 % en el navegador. No necesita Firebase, Supabase ni almacenamiento.

9. Ejemplo: documento generado localmente

Para cotizaciones, recibos o PDFs, el flujo recomendado es:

- Usuario introduce los datos.
- JavaScript valida y construye un modelo interno.
- El navegador genera la representación final.
- El archivo se descarga directamente.
- No se sube el PDF al servidor salvo que exista una función explícita de almacenamiento/compartición.

Esto convierte una operación que podría consumir almacenamiento cloud en una operación prácticamente local.  

10. Cómo utilizar Firebase

- Inicio de sesión y gestión de identidad.
- Sincronización sencilla de preferencias.
- Funciones serverless puntuales.
- Casos donde su ecosistema resulte más conveniente para una función concreta.

Firebase ofrece un plan sin coste con cuotas gratuitas para determinados servicios, pero las cuotas y condiciones pueden cambiar; deben verificarse en la documentación vigente antes de fijar límites de producto.

11. Cómo utilizar Supabase

- Datos relacionales que se beneficien de PostgreSQL.
- Consultas estructuradas y relaciones entre entidades.
- APIs y Edge Functions.
- Storage cuando realmente sea necesario.
- Funciones administrativas o analíticas que requieran SQL.

Supabase Free contempla actualmente límites concretos de base de datos, Storage, egress y Edge Functions. La aplicación debe vigilar esos límites y no diseñarse para consumirlos al máximo.

12. Seguridad mínima obligatoria

- Nunca colocar claves privadas, service-role keys o secretos en el frontend.
- Validar datos tanto en cliente como en servidor cuando exista backend.

- Aplicar autorización por usuario/registro.
- Limitar tamaño de payloads.
- Aplicar rate limiting a endpoints públicos.
- No almacenar información sensible si no es necesaria.
- Sanitizar contenido que vaya a renderizarse como HTML.
- Separar identificadores públicos de identificadores internos cuando sea conveniente.

❑ Regla: cualquier cosa que llegue al navegador debe considerarse potencialmente visible para el usuario. El frontend nunca es un lugar seguro para secretos.

13. ❑ SEO y descubrimiento

Cada herramienta útil puede tener una página propia y contenido que explique el problema, el cálculo y el resultado. La página debe aportar valor real, no ser una plantilla repetida con palabras cambiadas.

```
/herramientas/
/calculadora-precio-real/
/calculadora-autonomia-bateria/
/calculadora-regla-de-tres/
/generador-qr/
/compartir-texto-temporal/
/calculadora-materiales/
```

14. ❑ Modelo de monetización

- ❑ Gratis: herramientas esenciales.
- ❑ Publicidad: solo cuando exista tráfico suficiente y sea compatible con el canal de publicación.
- ❑ Premium: historial, sincronización, personalización y funciones avanzadas.
- ❑ B2B: paquetes de herramientas para pequeños negocios.
- ❑ Afiliación: únicamente cuando exista una relación natural y transparente con la herramienta.
- ❑ Herramientas especializadas: productos digitales con valor concreto.

15. ❑ Orden recomendado de construcción

Fase 0 — Especificación

Definir identidad, categorías, arquitectura, políticas de datos y criterios de calidad.

Fase 1 — Núcleo

Crear frontend, sistema de navegación, componentes UI, testing y estructura modular.

Fase 2 — 5 herramientas locales

Construir primero herramientas que no necesiten backend.

Fase 3 — Persistencia

Añadir una única función temporal con TTL y limpieza.

Fase 4 — Backend

Introducir Firebase o Supabase solo donde exista un caso de uso real.

Fase 5 — SEO

Crear páginas indexables, documentación y contenido útil.

Fase 6 — Monetización

Medir uso y añadir mecanismos de ingresos sin degradar la experiencia.

16. ☐ Checklist de aceptación de cada herramienta

- ☐ ¿Resuelve un problema concreto?
- ☐ ¿Puede ejecutarse localmente?
- ☐ ¿Realmente necesita guardar datos?
- ☐ Si guarda datos, ¿tienen TTL?
- ☐ ¿El usuario entiende qué se guarda?
- ☐ ¿Hay validación de entradas?
- ☐ ¿Funciona correctamente en Android?
- ☐ ¿Tiene estados de error y carga?
- ☐ ¿Tiene pruebas de casos normales y extremos?
- ☐ ¿La herramienta aporta valor original y suficiente?
- ☐ ¿Su coste operativo está controlado?
- ☐ ¿La monetización es compatible con la experiencia?

17. ☐ Principio maestro

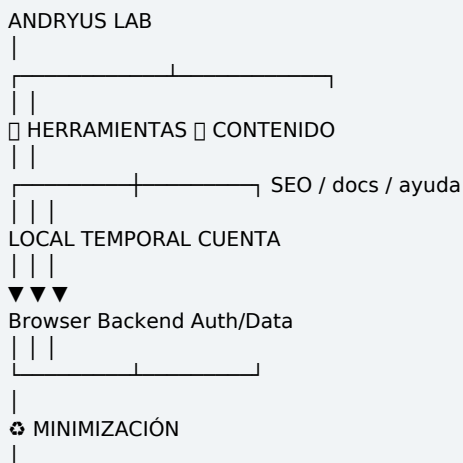
Andryus Lab no debe entenderse como “un sitio que almacena herramientas”. Debe entenderse como un motor de pequeñas soluciones. La infraestructura existe para habilitar esas soluciones, no para convertirse en el centro del producto.

La arquitectura ideal es aquella en la que 10.000 usuarios pueden utilizar una herramienta miles de veces sin que cada utilización obligue a guardar un archivo o una fila permanente. Esa es la razón de priorizar procesamiento local, datos mínimos y expiración.

18. ☐ Nota técnica sobre límites

Los límites gratuitos de GitHub Pages, Firebase y Supabase son condiciones de servicio y pueden cambiar. Antes de producción deben verificarse en la documentación oficial correspondiente. Este documento define una arquitectura resistente a esos límites; no debe interpretarse como garantía de cuotas permanentes.

19. ☐ Arquitectura objetivo resumida



20. □ Definición final del proyecto

Andryus Lab será una plataforma web modular, mobile-first y orientada a microherramientas. Su arquitectura priorizará procesamiento local, mínima persistencia, datos temporales con expiración y una separación estricta entre interfaz, lógica de dominio y servicios externos. Firebase y/o Supabase se incorporarán únicamente cuando una función requiera backend. El objetivo es mantener una infraestructura ligera, controlable y capaz de evolucionar hacia un producto económicamente sostenible.

□ **Siguiente documento recomendado: “Especificación Técnica v1.0” — definición del stack exacto, repositorio, variables de entorno, esquema de carpetas, contratos API, modelo de datos, reglas de seguridad, estrategia de TTL, CI/CD y primera herramienta funcional.**